

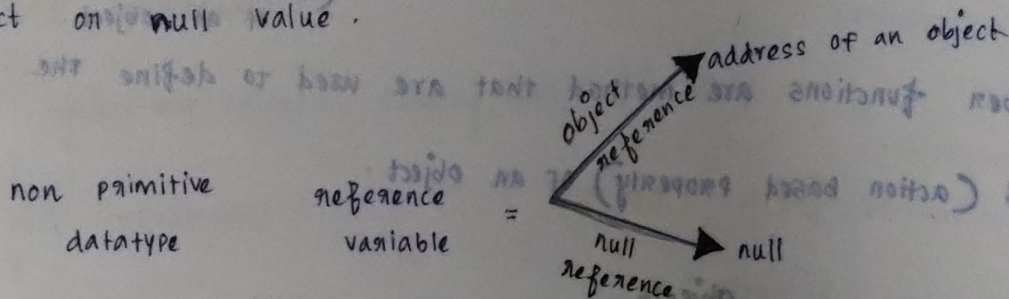
- Any variable that is declared using a non primitive

datatype is called as reference variable.

- Generally a reference variable is declared using class

name, interface name or enum name.

- A reference variable will contain either address of an object or null value.



NULL

- In java, NULL is a keyword
- NULL meaning is nothing
- NULL is the default value for all non primitive variables (reference variable)
- If a reference variable is assigned to null it is called as null references.

- If we try to access the non static members using null references it will cause NullPointerException.

```

ex)
class Actor
{
    void acting()
    {
        System.out.println("Actor is Acting");
    }
}
  
```

public class mainclass

```
{  
    public static void main(String[] args)
```

```
{  
    Actor a1 = new Actor();
```

```
    System.out.println(a1);
```

```
    a1.acting();
```

```
    Actor a2 = null; // null reference
```

```
    System.out.println(a2);
```

```
    a2.acting(); // Null pointer Exception
```

```
}
```

program. **Deep Copies**

```
class player
```

```
{  
    int JerseyNumber = 18;  
}
```

```
public class mainprogram
```

```
{  
    public static void main(String[] args)  
    {
```

```
        Player P1 = new Player();
```

```
        Player P2 = new Player();
```

```
        System.out.println(P1.JerseyNumber); //18
```

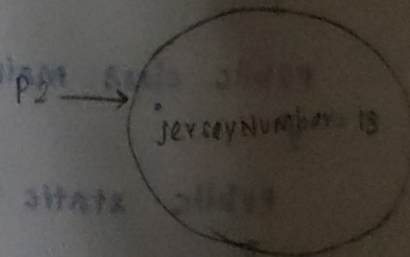
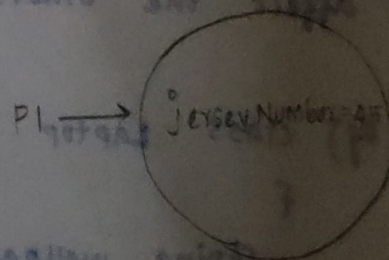
```
        System.out.println(P2.JerseyNumber); //18
```

```
        P1.JerseyNumber = 45;
```

```
        System.out.println(P1.JerseyNumber); //45
```

```
        System.out.println(P2.JerseyNumber); //18  
    }
```

```
}
```



Deep copies Vs shallow copies

Deep copies

- Multiple references variable of the same class with each reference variable assigned to a distinct object. is called as deep copies
- In case of deep copies changes made by one reference variable will not affect the others.

Shallow copies

- Multiple references variables of the same class ^{which are} assigned to a common object is called as shallow copies
- In case of shallow copies changes made by one reference variable will affect the others.

eg) class Laptop

{

String wallpaper = "Alia.png";

}

public class MainClass12

{

public static void main(String[] args)

{

Laptop l1 = new Laptop();

Laptop l2 = l1;

System.out.println(l1.wallpaper); // Alia.png

System.out.println(l2.wallpaper); // Alia.png

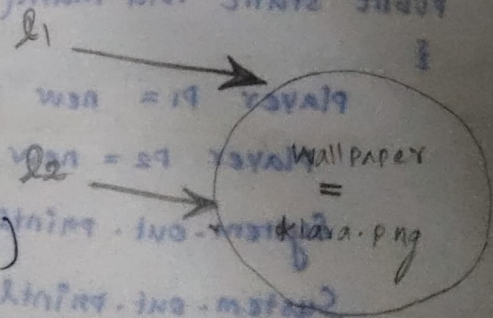
l1.wallpaper = "Kiran.png";

System.out.println(l1.wallpaper); // Kiran.png

System.out.println(l2.wallpaper); // Kiran.png

}

}



Deep copies

each references variable is assigned to a distinct object

changes made by one references variables will not affect the others

Shallow copies

each references variable is assigned to a common object.

changes made by one references variables will affect the others.